CHAPTER 4

ADAPTIVE GRAPH RETRIEVAL FOR DYNAMIC CLASSIFICATION WITH GORAG

This chapter is based on *GORAG: Graph-based Online Retrieval Augmented Generation for Dynamic Few-shot Social Media Text Classification*, published in the Proceedings of the ACM Web Conference 2026 [1]. It studies the selection and maintenance of label evidence when a classification task acquires new classes over several rounds. The central question is how a language model can receive a query-specific candidate set while the evidence connecting text to labels changes over time.

4.1 Dynamic Few-shot Classification

A classification label defines a decision that an application asks a model to make. When the application adds a more specific category, the decision changes even if the underlying language model stays fixed. For example, a social media classification task can initially distinguish threatening language and later introduce identity insults. A model must then distinguish the new label from earlier labels using the available annotated examples. The difficulty concerns both learning the meaning of a new category and keeping the current alternatives visible during prediction.

Few-shot classification supplies a small number of labeled examples for each class. Dynamic few-shot classification adds classes in successive rounds [2]. The size of the label universe therefore grows independently of the number of examples available for each label. A retrieval system must update its evidence source as classes arrive, and each prediction must refer to the label set available at that round. This setting differs from repeatedly evaluating a fixed classifier on an unchanged collection of labels.

The task also exposes a limitation of simply placing more material in a language model's context. Additional examples provide information about labels, but a larger context can contain many examples that have little bearing on the current text. The candidate list itself grows as classes accumulate. Selecting text passages without narrowing the associated label alternatives leaves the model with a long classification instruction. GORAG therefore treats the selection of candidate labels as an explicit retrieval problem.

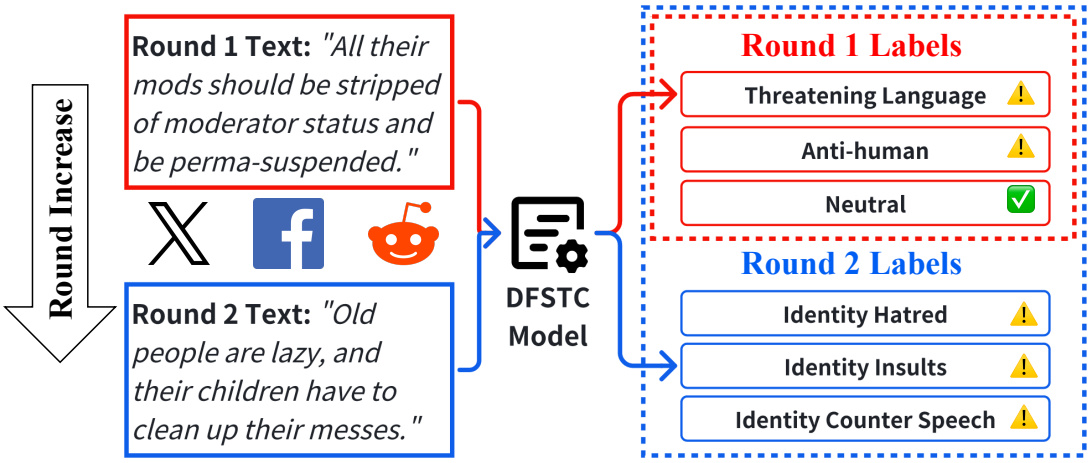


Figure 4.1: Two rounds of the dynamic classification task. New abuse categories extend the available labels. Reproduced from Figure 1 of GORAG [1].

The index used in this chapter is a weighted graph of keywords and labels. A keyword can occur in examples from several classes. Its incident edges record these associations, and their costs distinguish how the term is represented in the indexed corpus. The graph allows several query keywords to be considered jointly. A label can participate in the connection between query keywords even when examining one keyword in isolation would give an ambiguous set of alternatives. The retrieved label set depends on the query terminals and the current graph.

GORAG also updates its index after classifying an unlabeled query. Keywords that were absent from the index can then become available to later queries. The associated label is the model's prediction, which introduces an important distinction between supervised initialization and subsequent online expansion. The system receives true labels for the few-shot support examples; it does not receive the true label of a query before predicting that query. This information boundary defines the interpretation of the online evaluation.

The research question has three connected parts. First, how should a small support set become a graph whose edges distinguish keyword–label associations? Second, how should the graph select a different candidate set for each text as the label universe expands? Third, how can observations from previously classified texts enlarge the index? The following sections develop these operations and then examine their reported classification and resource results.

4.1.1 Rounds and information availability

Let $\mathcal{Y}^r_{\text{new}}$ denote the labels introduced at round r. The current label universe is

$$\mathcal{Y}^r = \bigcup_{i=1}^{r} \mathcal{Y}^i_{\text{new}}. \tag{4.1}$$

The few-shot support data provide k annotated texts per class, with k equal to 1, 5, or 10 in the main experiments. A query text t must be assigned a label from $\mathcal{Y}^r$. The superscript r identifies the current task state; it is not a model layer or a reasoning step within one prediction.

For clarity, the support collection visible at a round and the current query stream are separate objects. Support texts carry supplied labels. Query texts become visible as the system processes them, and their predictions can be used for subsequent index updates. Corpus statistics used in keyword scoring refer to training and query texts seen so far. Maintaining these statistics does not provide the query's correct class. The online association nevertheless inherits prediction errors when the classifier chooses an incorrect label.

Semantic label names supply another source of information. Where names are available, GORAG uses a language model to generate representative descriptions or keywords for labels. These descriptions enter the classification context, and label-derived keywords can support graph initialization. The zero-shot experiment later in this chapter uses this source without any labeled examples. It should therefore be read as a distinct initialization protocol rather than as a smaller support sample from the few-shot experiment.

4.2 Weighted Graph Indexing

GORAG maintains an undirected weighted graph $\mathcal{G}^r = (\mathcal{V}^r, \mathcal{E}^r, \mathcal{W}^r)$. Its node set contains keyword nodes and label nodes. A keyword node represents an extracted word or short phrase. A label node represents a target class. The representation places label alternatives inside the same structure that stores their lexical associations. Individual support documents are used to construct this representation; they are not defined as a third category of exemplar nodes in the graph.

A keyword–label edge is introduced when a keyword occurs in a text associated with that label. Repeated occurrences across texts contribute to the edge weight. A keyword shared by several classes can therefore have several label neighbors, with a different cost for each association. Query retrieval will use these costs when choosing connections between mapped keywords. The graph does not encode a fixed rule that every occurrence of a keyword implies the same class.

This representation separates the unit used to store evidence from the unit used to make the final prediction. The graph stores keyword–label associations. The language model receives the original query text as well as the retrieved alternatives and their descriptive information. The model can use syntactic and contextual information from the whole query even when retrieval has operated on a smaller collection of keywords. The keyword-only ablation examines the

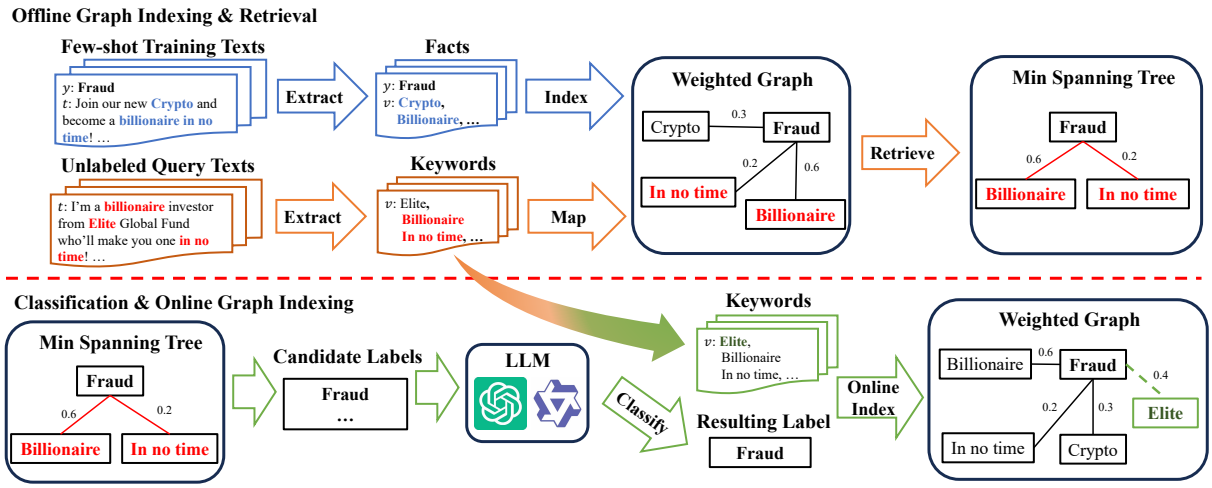


Figure 4.2: GORAG's offline indexing, query-dependent retrieval, classification, and online update. New query keywords are associated with the predicted class after classification. Reproduced from Figure 2 of GORAG [1].

consequence of removing this full-text component.

4.2.1 Keyword extraction and correlation scores

GORAG scores terms using normalized TF–IDF [3]. Its extraction vocabulary includes words and phrases of fewer than three words. Let $\mathcal{T}_{\leq r}$ denote the corpus seen so far, and let $\mathrm{df}(v)$ be the number of its texts containing term v. The reported correlation score is

$$CS(v,t) = \frac{\mathrm{count}(v,t)}{|t| \log\left(|\mathcal{T}_{\leq r}| + 1\right)} \log \frac{|\mathcal{T}_{\leq r}| + 1}{\mathrm{df}(v) + 1}. \tag{4.2}$$

Here $\mathrm{count}(v,t)$ is the occurrence count of v in t, and $|t|$ is text length. The expression combines frequency within the text with frequency across the observed corpus. Its corpus term changes when additional texts are observed. Terms with $CS(v,t) > \tau$ are retained, with $\tau = 0.5$ in the original study. The method uses the score both to select keywords and to calculate association costs.

A statistical extraction score gives each retained term a numerical contribution before graph retrieval. This contribution has a different role from the final language model's confidence in a label. The graph does not obtain a supervised probability of classification correctness from this formula. Instead, the score determines which lexical units are indexed and how expensive their associations will be in the retrieval structure. The evaluation of the full pipeline is therefore needed to establish the effect on classification.

Using short phrases can preserve a lexical combination that would be split by unigram index-

ing. At the same time, the extraction rule restricts what becomes a terminal later. Content absent from the extracted keyword set cannot directly act as a retrieval terminal for that query. The full query text remains available to the classifier, but its access to label evidence has already been shaped by the index representation. This relationship explains why extraction and classification should be described as separate stages.

4.2.2 Keyword–label edge costs

For keyword v and label y, let $\mathcal{T}^{r,v,y}$ be the seen texts that contain v and are associated with y. GORAG assigns the edge cost

$$w^r_{v,y} = \frac{\sum_{t_j \in \mathcal{T}^{r,v,y}} (1 - CS(v, t_j))}{|\mathcal{T}^{r,v,y}|}. \tag{4.3}$$

The complement converts a larger correlation score into a smaller contribution to connection cost. Averaging makes the edge depend on all contributing observations rather than on one occurrence. Offline observations use the supplied class of a support text; online observations use a predicted class when new query keywords are inserted.

An edge weight thus belongs to a particular keyword–label pair. It is not a global importance weight assigned to the keyword irrespective of class. The same keyword may occur in several label contexts, and the corresponding averages can differ. Retrieval uses the resulting cost structure together with connectivity. Replacing every cost by one preserves the graph's adjacency while removing this distinction, which is the purpose of the unit-weight ablation.

The averaging rule also identifies the granularity at which evidence is combined. Texts are not concatenated into one label description before the edge is scored. Each contributing text supplies a correlation value and the edge records their mean complement. A later text can contribute another occurrence to the associated pair. This design lets the index summarize recurrent lexical associations without putting every support text into each classification prompt.

4.2.3 Merging label rounds

At each round, the graph created from newly supplied information is merged with the graph accumulated from earlier rounds. The node and edge sets are unions. This operation retains the label universe already introduced while adding nodes and associations for the new task state. It differs from rebuilding a fresh graph containing only the most recently introduced labels, which would remove access to earlier alternatives.

GORAG also introduces edges between newly introduced and earlier label nodes. These bridges support connections between parts of the graph indexed in different rounds. Let $\mathscr{M}_y^r$ denote the keyword neighbors of label y. For a new label y_n and an earlier label y_o, GORAG reports

$$w^r_{y_n,y_o} = \frac{1}{2}\left(\frac{\sum_{v\in\mathscr{M}^r_{y_n}} w^r_{v,y_n}}{2|\mathscr{M}^r_{y_n}|} + \frac{\sum_{v\in\mathscr{M}^r_{y_o}} w^r_{v,y_o}}{2|\mathscr{M}^r_{y_o}|}\right). \tag{4.4}$$

The formula uses costs incident to both labels. These factors determine the relative cost of traversing a label bridge rather than a keyword association.

Label bridges have a different evidential meaning from keyword–label edges. A keyword edge records a term's occurrence in texts associated with a class. A label bridge is added by the graph-merging procedure. It provides a connection through which retrieval can move between label regions. Consequently, a retrieved path should be interpreted as an operation on the constructed index, with these distinct edge semantics, rather than as a sequence of independently observed real-world relations.

4.3 Adaptive Candidate Label Retrieval

For a query t, GORAG extracts keywords using the indexing procedure and partitions them according to whether they already occur in $\mathscr{V}^r$. Write these sets as $\mathscr{V}^t_{\text{exist}}$ and $\mathscr{V}^t_{\text{notexist}}$. The existing keywords become the terminals for current retrieval. The unseen keywords are held for online insertion after classification. The partition prevents the retrieval operation from treating an unindexed term as if it already had a known label neighborhood.

Retrieval seeks a low-cost connected structure that contains the mapped terminals. Label nodes encountered in this structure form the candidate set $\widehat{\mathscr{Y}}^r_t$. Connecting several terminals can select intermediate labels and keywords. The graph therefore determines the number of candidate labels through the returned structure, rather than requiring the same number of labels for all queries. A short query and a query containing several mutually distant indexed terms can produce different retrieval structures.

The distinction between terminals and intermediate nodes is central. Terminals must participate because they represent the query's mapped evidence. Intermediate nodes are included because the selected connections pass through them. The final candidate labels are the label nodes among these selected vertices. A label does not enter the candidate set merely because it is present somewhere in the full index. This selective use of graph topology is how the retrieved classification instruction can be smaller than the current task's complete label universe.

This terminal-connecting objective is related to the Steiner tree problem. In that problem, the required terminals are a subset of graph vertices, and the solution may use additional vertices to connect them. An ordinary minimum spanning tree instead spans all vertices of its input graph. These objects play distinct roles in GORAG's procedure. Describing every intermediate structure as an exact optimum for the original terminal problem would obscure which vertices are required and which connections have been chosen by the heuristic.

4.3.1 Tree and auxiliary-graph operations

GORAG uses a greedy terminal-connecting procedure based on Mehlhorn's algorithm [4]. The mapped keywords serve as terminals. An auxiliary graph H represents connections between terminals, with edge weights given by path costs. A minimum spanning tree of H selects a collection of terminal connections. Expanding these connections into their underlying paths produces the graph structure used for candidate selection. The language model then receives the labels encountered in this structure.

The auxiliary tree and the returned structure have different vertex sets. The auxiliary tree expresses connections among the terminals. Their expanded paths can also contain keyword and label nodes that were not terminals. These intermediate nodes provide the connection between the query's lexical evidence and the candidate classes. The terminal set changes from query to query, while the costs reflect the graph accumulated at the current round.

The retrieval procedure favors a connected explanation of the mapped keywords over independently selecting a fixed number of labels for each keyword. A label may be included because it connects several terms, and an additional keyword node may connect terms that have no direct edge. The returned structure can therefore express joint evidence that would be lost if each terminal were treated as an independent lookup. Its label intersection is the compact output used by classification.

Once the expanded structure S_t^r is obtained, the candidate set is

$$\hat{\mathcal{Y}}_t^r = \mathcal{V}(S_t^r) \cap \mathcal{Y}^r. \tag{4.5}$$

This expression makes the retrieval output explicit. The selected structure is an intermediate object; the label intersection is the interface needed by the classifier. Supporting keywords and label descriptions then provide additional context for these alternatives.

The adaptive property concerns candidate selection after the query has been mapped. It does not remove every parameter from the system. Keyword extraction still uses a score and

an extraction rule. The language model and its prompts also remain part of the configuration. The useful distinction is between determining a candidate set from the query's connections and applying a fixed retrieval cutoff to all texts.

4.4 Classification and Online Indexing

GORAG constructs a classification input by concatenating the query text, extracted query keywords, retrieved candidate labels, and representative keywords associated with those labels. With $\mathcal{K}^t$ denoting query keywords and $\mathcal{K}_{\widehat{\mathcal{Y}}_t^r}$ the representative label information, the input is

$$c_t = \text{Concat}\left(t, \mathcal{K}^t, \widehat{\mathcal{Y}}_t^r, \mathcal{K}_{\widehat{\mathcal{Y}}_t^r}\right). \tag{4.6}$$

A language model applies the classification instruction p_{classify} to this input and produces y_t^*. The graph narrows and describes the alternatives; the language model performs the final text classification.

Keeping the original query in this interface matters because keyword extraction removes ordering, local syntax, and surrounding discourse from the retrieval representation. The selected candidate set does not by itself determine how these features should affect the decision. The classifier can inspect the complete query alongside the retrieved label information. GORAG's keyword variant removes the full query from this input to test the contribution of that context under the same overall framework.

After the prediction, each keyword in $\mathcal{V}^t_{\text{notexist}}$ is inserted into the graph and connected to y_t^*. The update can be written as

$$\mathcal{V}^r \leftarrow \mathcal{V}^r \cup \mathcal{V}^t_{\text{notexist}}, \tag{4.7}$$

$$\mathcal{E}^r \leftarrow \mathcal{E}^r \cup \{(v, y_t^*) : v \in \mathcal{V}^t_{\text{notexist}}\}. \tag{4.8}$$

New associations receive weights through the reported weighting mechanism. Later queries can then map terms that were previously absent from the index. The update changes the retrieval source without fine-tuning the language model's parameters.

The prediction precedes the update. A query therefore supplies its text to the current prediction, while its predicted class contributes to future index state. A mistaken prediction can attach a new keyword to an incorrect label. Edge weighting determines the cost of using that association, but it is not a mechanism for obtaining missing ground truth. The online ablation

measures the overall contribution of using this expanding source under the reported protocol.

This distinction also determines what an evaluation result measures. Online accuracy includes decisions made with an index that has been affected by earlier predictions. It is a result for a stateful retrieval procedure. Evaluating queries independently against an unchanged support graph would answer a different question. The offline variant provides one such comparison by suppressing online expansion while retaining the rest of the method's components.

4.4.1 Computational organization

The graph separates work performed at a label update from work performed for each query. Offline indexing processes the supplied texts, creates or updates keyword associations, generates label information when available, and merges the round's graph. The cross-round label bridges introduce interactions between the newly indexed and earlier label regions. In the source's abstract operation count, construction is expressed as

$$O\big(|\mathcal{Y}^r||\mathcal{Y}^{r-1}| + |\mathcal{V}^r_{\text{new}}||\mathcal{D}^r|\big)\,. \tag{4.9}$$

The two terms account for label merging and keyword–text processing. This count treats the associated text-processing and graph operations at the granularity used in that analysis. The number of tokens processed by a language-model call depends on the prompt and generated output, so the empirical token study supplies a separate measurement of model-facing work.

Per-query processing maps the extracted terms to the existing graph, obtains a connecting structure, builds the classification prompt, and inserts new terms after prediction. The index state can therefore grow during one round as well as between label rounds. Graph memory and prompt length measure different consequences of that growth. A larger stored index can provide more candidate associations while a query still receives only the labels selected by its own retrieval structure. The resource experiments report these two quantities separately.

4.5 Experimental Design

The published study evaluates CAD, WOS, and IFS-Rel. CAD contains social media posts annotated for contextual abuse [5]. WOS contains academic texts associated with research-area labels [6]. IFS-Rel supplies relation-classification examples derived from news text in an incremental few-shot benchmark [2]. These datasets exercise different label vocabularies and text lengths within the dynamic classification setting.

Table 4.1: Dataset statistics for the four-round experiments, transposed from the original table. IFS-Rel has 40 test samples per label. Round label counts refer to newly introduced labels.

Statistic	CAD	WOS	IFS-Rel
Average text tokens	154	200	105
Round 1 test texts	240	2,417	640
Round 1 new labels	4	32	16
Round 2 test texts	60	2,842	640
Round 2 new labels	3	53	16
Round 3 test texts	286	2,251	640
Round 3 new labels	3	30	16
Round 4 test texts	231	1,886	640
Round 4 new labels	3	18	16
Total test texts	817	9,396	2,560

The four-round split introduces 13 labels across CAD's rounds, 133 across WOS, and 64 across IFS-Rel. The test collections contain 817, 9,396, and 2,560 examples, respectively. IFS-Rel uses 40 test examples per label. Its balanced test allocation differs from the changing class frequencies in the other collections. Accuracy should therefore be interpreted with each dataset's actual split and distribution.

The label counts in Table 4.1 describe labels introduced in each round. The classifier operates over their cumulative union. Thus the second WOS round adds 53 labels to the 32 labels already introduced, rather than replacing the earlier set by a new 53-label task. This distinction matters when relating the dataset statistics to the size of the classification instruction and to the graph state retained from earlier rounds.

The main few-shot study uses 1, 5, and 10 labeled examples per class. CAD and WOS RAG experiments use Qwen2.5-7B, while IFS-Rel uses GPT-4o. These are separate backbone conditions. The source reports an Intel Xeon Gold 5220R CPU at 2.20GHz and a single NVIDIA A100-SXM4-40GB GPU for the experiments. Runtime observations concern this implementation and hardware setting; API-backed inference also reflects the service used during the original study.

Classification accuracy requires an exact match between the model output and the ground-truth label name or number. An output that cannot be matched to the required class is not counted as correct. This protocol evaluates the final classification decision rather than a graded semantic similarity between arbitrary generated answers. A fluent explanation alone does not satisfy the metric.

Table 4.2: WOS classification accuracy with 1 support examples per label. Qwen2.5-7B is the RAG backbone. Values and emphasis reproduce the corresponding columns of GORAG's main table.

Method	Round 1	Round 2	Round 3	Round 4
Entailment	0.3695	0.3994	0.4510	0.4030
NaiveRAG	0.3885	0.2267	0.1821	0.1653
LongLLMLingua	0.3806	0.2155	0.1770	0.1468
GraphRAG	0.3852	0.2213	0.1816	0.1641
LightRAG	0.3930	0.2202	0.1743	0.1625
HippoRAG2	0.4133	0.2552	0.2205	0.1906
GORAG	**0.4866**	**0.4790**	**0.4736**	**0.4420**

4.5.1 Compared methods and controls

The study compares six baselines. Entailment constructs text–label entailment pairs and fine-tunes a classifier [2]. NaiveRAG retrieves text chunks from the indexed support information. Retrieved context is compressed by LongLLMLingua before classification [7]. GraphRAG, LightRAG, and HippoRAG2 supply graph-based retrieval alternatives [8–10]. Their classification results are measurements of the adaptations used in this study.

Entailment uses RoBERTa-large for five training epochs, a batch size of 16, a learning rate of 10^{-6}, and Adam. LongLLMLingua is evaluated with compression rates of 0.75, 0.8, and 0.85, with 0.8 selected in the reported comparison. GraphRAG and LightRAG use local search. The source uses the nano-graphrag implementation for GraphRAG and NaiveRAG and the other baselines' official implementations. These choices identify the systems that produced the reported results.

The baseline collection tests several ways of allocating external context. NaiveRAG selects passages. LongLLMLingua changes which tokens remain in that context. The graph baselines select graph-associated information. GORAG explicitly selects labels through a connection structure and expands its lexical index online. These differences motivate the comparison, while the ablations isolate changes within GORAG's own pipeline more directly.

4.6 Classification Results

The tables beginning with Tables 4.2 and 4.5 preserve the WOS and IFS-Rel portions of the main accuracy table. Each dataset is split into one-, five-, and ten-shot tables, with rounds as columns. Each value is a fractional accuracy, and the labels available to the model expand with the round. The tables should be read by comparing methods under the same dataset, round, support size, and backbone.

Table 4.3: WOS classification accuracy with 5 support examples per label. Qwen2.5-7B is the RAG backbone. Values and emphasis reproduce the corresponding columns of GORAG's main table.

Method	Round 1	Round 2	Round 3	Round 4
Entailment	0.3823	0.4471	0.4857	0.4387
NaiveRAG	0.3904	0.2154	0.1475	0.1556
LongLLMLingua	0.3823	0.2202	0.1567	0.1382
GraphRAG	0.3897	0.2197	0.1770	0.1634
LightRAG	0.3806	0.2216	0.1767	0.1626
HippoRAG2	0.4245	0.2656	0.2210	0.1860
GORAG	**0.4990**	**0.5109**	**0.4996**	**0.4717**

Table 4.4: WOS classification accuracy with 10 support examples per label. Qwen2.5-7B is the RAG backbone. Values and emphasis reproduce the corresponding columns of GORAG's main table.

Method	Round 1	Round 2	Round 3	Round 4
Entailment	0.4187	0.4222	0.4787	0.4442
NaiveRAG	0.3897	0.2187	0.1799	0.1649
LongLLMLingua	0.3901	0.2198	0.1608	0.1493
GraphRAG	0.3906	0.2219	0.1786	0.1625
LightRAG	0.3815	0.2145	0.1799	0.1632
HippoRAG2	0.4208	0.2666	0.2242	0.1915
GORAG	**0.5134**	**0.5284**	**0.5234**	**0.4929**

Table 4.5: IFS-Rel classification accuracy with 1 support examples per label. GPT-4o is the RAG backbone. Values and emphasis reproduce the corresponding columns of GORAG's main table.

Method	Round 1	Round 2	Round 3	Round 4
Entailment	0.3391	0.4008	0.3061	0.2971
NaiveRAG	0.7343	0.5394	0.4719	0.5233
LongLLMLingua	0.8750	0.5734	0.4703	0.5819
GraphRAG	0.6406	0.4406	0.3828	0.4297
LightRAG	0.7578	0.4906	0.3802	0.4531
HippoRAG2	0.7313	0.4875	0.4297	0.4984
GORAG	**0.9688**	**0.5938**	**0.4844**	**0.6165**

Table 4.6: IFS-Rel classification accuracy with 5 support examples per label. GPT-4o is the RAG backbone. Values and emphasis reproduce the corresponding columns of GORAG's main table.

Method	Round 1	Round 2	Round 3	Round 4
Entailment	0.6046	0.5516	0.4703	0.4395
NaiveRAG	0.7344	0.5406	0.4719	0.4938
LongLLMLingua	0.8906	0.5922	0.4741	0.4903
GraphRAG	0.7813	0.4688	0.3734	0.4422
LightRAG	0.8281	0.5155	0.4010	0.4804
HippoRAG2	0.7375	0.5016	0.4328	0.5008
GORAG	**0.9688**	**0.6172**	**0.4948**	**0.4988**

Table 4.7: IFS-Rel classification accuracy with 10 support examples per label. GPT-4o is the RAG backbone. Values and emphasis reproduce the corresponding columns of GORAG's main table.

Method	Round 1	Round 2	Round 3	Round 4
Entailment	0.5859	0.5828	0.5193	0.4895
NaiveRAG	0.7406	0.5469	0.4688	0.5000
LongLLMLingua	0.9219	0.5806	0.5225	0.5038
GraphRAG	0.8125	0.4719	0.3141	0.3922
LightRAG	0.8594	0.5156	0.4427	0.4610
HippoRAG2	0.7406	0.4938	0.4438	0.4953
GORAG	**0.9688**	**0.6250**	**0.5344**	**0.5469**

On WOS with one support example per label, GORAG reports accuracies of 0.4866, 0.4790, 0.4736, and 0.4420 across the four rounds. Entailment reports 0.3695, 0.3994, 0.4510, and 0.4030 in the same columns. The corresponding GraphRAG values are 0.3852, 0.2213, 0.1816, and 0.1641. These comparisons locate the result in a setting where the number of candidate labels increases substantially over the evaluation.

Additional support examples improve GORAG's WOS accuracy in the main table. At the fourth round, its one-, five-, and ten-shot values are 0.4420, 0.4717, and 0.4929. At the first round they are 0.4866, 0.4990, and 0.5134. The changing label universe and changing support size are separate experimental axes. Comparing the fourth-round ten-shot score to the first-round one-shot score would mix these conditions.

IFS-Rel uses GPT-4o. In its first round, GORAG reports 0.9688 for all three support sizes, while the fourth-round scores are 0.6165, 0.4988, and 0.5469. The five-shot fourth-round value is below the one-shot value, so increasing support size does not produce a monotonic improvement in every setting. LongLLMLingua reports 0.5819, 0.4903, and 0.5038 in these fourth-round columns. HippoRAG2 reports 0.4984, 0.5008, and 0.4953; its five-shot result exceeds GORAG's five-shot result in that column.

CAD uses Qwen2.5-7B. Its GORAG first-round accuracy is 0.6125 in all three support settings. The fourth-round one- and five-shot accuracies are 0.3133 and 0.3146. These values show that adapting the retrieval mechanism does not remove the difficulty of later rounds. The relevant empirical observation is the comparison with the other methods under those conditions, rather than a claim that the task's accuracy remains constant as labels accumulate.

The results establish different patterns on different datasets. WOS has many labels and longer academic texts. IFS-Rel uses a different backbone and balanced test allocation. CAD concerns contextual abuse categories. A single pooled improvement would combine these task and model differences. The chapter instead uses the per-condition scores to examine whether query-specific

Table 4.8: Ablation experiments on WOS for GORAG's variants.

Dataset	Model	Round			
		R1	R2	R3	R4
WOS	GORAG *offline&unit*	0.2979	0.2287	0.2645	0.2249
	GORAG *offline*	0.3063	0.2302	0.2455	0.2156
	GORAG *unit*	0.4706	0.4394	0.4407	0.3899
	GORAG *keyword*	0.4746	0.4606	0.4455	0.4030
	GORAG	**0.4862**	**0.4649**	**0.4814**	**0.4210**

label selection and online evidence maintenance help the evaluated classifier.

4.7 Ablations and Resource Use

The WOS ablation study uses a one-shot Qwen2.5-7B setting. The offline variant disables online indexing. The unit variant assigns cost one to every graph edge. The offline-and-unit variant removes both features. The keyword variant uses the extracted keywords in place of the complete query text for classification. These variants change specific inputs or index operations while retaining GORAG's overall classification task.

In the ablation table, the full method reports 0.4862, 0.4649, 0.4814, and 0.4210 across rounds. The offline variant reports 0.3063, 0.2302, 0.2455, and 0.2156. The unit variant reports 0.4706, 0.4394, 0.4407, and 0.3899. Under this ablation protocol, removing online insertion has a larger observed effect than replacing the weighted graph with unit costs. This comparison concerns the variants reported together in that table.

The offline-and-unit variant reports 0.2979, 0.2287, 0.2645, and 0.2249. Its third- and fourth-round values exceed those of the offline variant. The effects of removing these components are therefore not strictly additive across rounds. The keyword variant reports 0.4746, 0.4606, 0.4455, and 0.4030, below the full method in each displayed column. The full query provides information useful to the classifier in these experiments even though keywords determine the retrieval terminals.

The full-method row of this ablation table differs from the full-method row of the main WOS table. They are retained as separate reported result series. Component comparisons above use only the ablation table's full-method row. Using a main-table value as the control for an ablation value would introduce an unexplained change of result series.

Table 4.9: Token cost on IFS-Rel dataset with 1-shot setting.

Model	AVG Token per Query	AVG Cost per Query (USD)
GPT-4o	634.6	$0.005
LongLLMLingua	2771.3	$0.021
NaiveRAG	4537.4	$0.034
LightRAG	7866.3	$0.059
GraphRAG	9296.3	$0.070
GORAG	**557.5**	**$0.004**

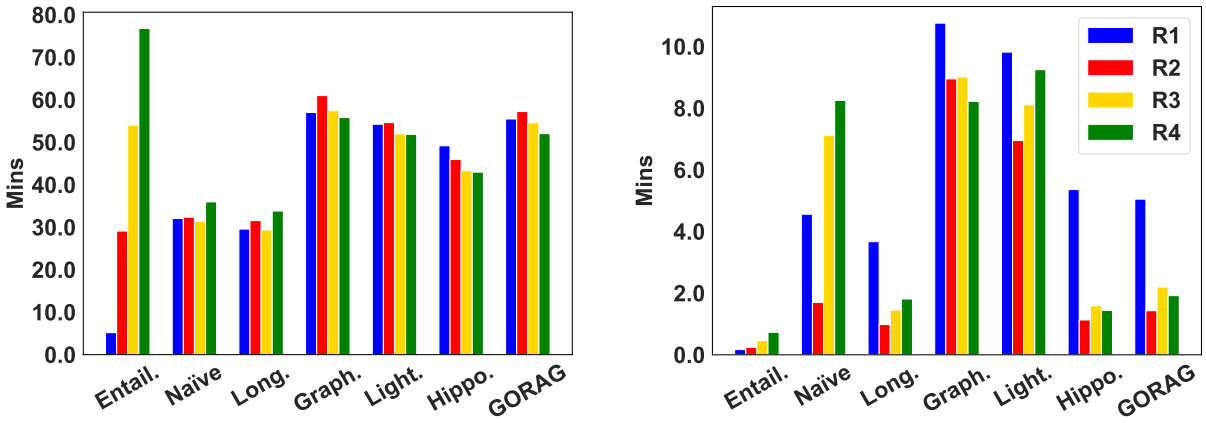
4.7.1 Prompt tokens and graph growth

The IFS-Rel one-shot token study reports average tokens per query and the corresponding US-dollar costs under the original study's pricing conditions. GORAG uses 557.5 tokens per query and reports a cost of $0.004. The direct GPT-4o comparison uses 634.6 tokens and reports $0.005. LongLLMLingua uses 2,771.3 tokens, NaiveRAG 4,537.4, LightRAG 7,866.3, and GraphRAG 9,296.3. Their reported costs are $0.021, $0.034, $0.059, and $0.070, respectively.

The token comparison measures how much context reaches the model in the evaluated implementation. GORAG's candidate-label representation uses fewer tokens than the listed retrieval baselines in this experiment. The historic dollar values provide the cost calculation reported with those measurements; they are not current API price quotations. Token totals and monetary costs should remain attached to this IFS-Rel one-shot protocol when discussing efficiency.

Graph growth is measured separately during the first IFS-Rel round with GPT-4o. The reported index contains 1,644 nodes and 1,771 edges at the 25-percent checkpoint, and 5,299 nodes and 5,503 edges at the final checkpoint. The corresponding reported memory values are 13.3 KB and 46.4 KB. These measurements concern the graph representation in that experiment. They do not include the complete memory required to host or serve the language model.

The original runtime study compares total time on WOS and CAD under the one-shot setting, using Qwen2.5-7B. Its figures belong to those dataset and hardware conditions. Online insertion adds work after each prediction, while statistical keyword indexing and smaller model inputs can change other parts of the pipeline. Runtime must therefore be measured end to end rather than inferred from token counts alone. This chapter retains the separation between prompt-token evidence, graph-memory evidence, and the source's total-time experiment.



(a) WOS. (b) CAD.

Figure 4.3: Total time in minutes for the one-shot four-round experiments with Qwen2.5-7B. The original R1–R4 legend in panel (b) applies to both panels. Reproduced from Figure 3 of GORAG [1].

Table 4.10: 0-shot experiments for GORAG and RAG baselines on IFS-Rel dataset under 0-shot setting.

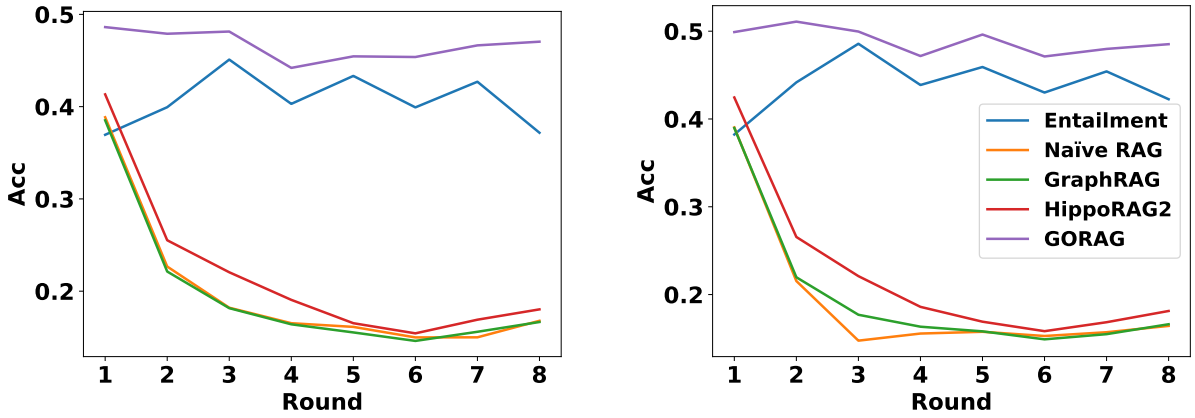
Model	**R1**	**R2**	**R3**	**R4**
GPT-4o [11]	0.9219	0.5109	0.4063	0.5936
LongLLMLingua [7]	0.9281	0.5297	0.4234	0.5806
LightRAG [9]	0.6718	0.3047	0.2135	0.0977
GraphRAG [8]	0.5625	0.1875	0.1458	0.0938
GORAG	**0.9328**	**0.5406**	**0.4375**	**0.6063**

4.8 Zero-shot and Round Generalization

The zero-shot IFS-Rel experiment supplies label names and language-model-generated descriptions without annotated support texts. GORAG extracts keywords from these descriptions to initialize its graph. The experiment consequently tests whether description-derived associations can support candidate selection and subsequent classification. It does not measure a classifier with no external label information.

GORAG reports zero-shot accuracies of 0.9328, 0.5406, 0.4375, and 0.6063. The direct GPT-4o comparison reports 0.9219, 0.5109, 0.4063, and 0.5936. LongLLMLingua reports 0.9281, 0.5297, 0.4234, and 0.5806. The graph-based comparisons in the same table are lower under this initialization protocol. These values support the use of label descriptions as an initial retrieval source for this experiment.

The zero-shot result also illustrates why support size is not the only determinant of graph content. A graph can be initialized from label descriptions, support texts, or a combination allowed by the protocol. The few-shot results and zero-shot results use different available infor-



(a) One-shot.

(b) Five-shot.

Figure 4.4: WOS classification across the reported round splits. Points 1–4 use the four-round split, 5–6 the six-round split, and 7–8 the eight-round split. The legend in panel (b) applies to both panels. Reproduced from Figure 4 of GORAG [1].

mation. The zero-shot and few-shot comparisons therefore evaluate different graph initialization sources.

The round-generalization study uses WOS split into four, six, and eight rounds under one- and five-shot settings. The displayed curve combines rounds one to four from the four-round split, rounds five and six from the six-round split, and rounds seven and eight from the eight-round split. These points represent results from different partitions of the label set. They do not form the trajectory of one index updated through eight consecutive rounds.

This construction lets the study display behavior at later rounds while retaining a sufficient test collection in each split. Its interpretation depends on the partitioning protocol. A later point can differ because it has a different current label universe and because its preceding rounds used another partition. The appropriate conclusion concerns the reported performance across these split settings, with the original curves providing the per-round evidence.

4.8.1 A retrieved classification case

Figure 4.5 reproduces a contextual-abuse example from the original study. The query contains the terms highlighted in red. These terms map to keyword nodes, and the selected connecting structure passes through an insult label. The classifier outputs the abuse category shown in the figure. This example exposes the relationship between the extracted query evidence, the intermediate label node, and the final classification.

The useful property of the example is that candidate selection has a visible provenance. The label is connected to more than one term in the indexed representation. The displayed

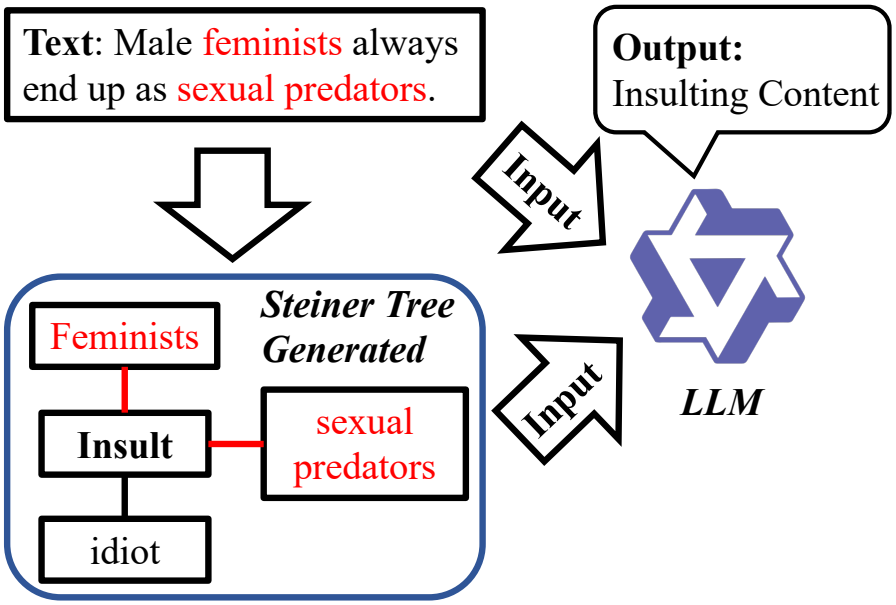


Figure 4.5: A contextual-abuse example and its selected graph connections. Reproduced from Figure 5 of GORAG [1].

nonterminal neighbor also shows that a keyword neighborhood can contain more information than the connection chosen for the current text. Selecting a structure distinguishes the evidence used to connect the query terms from other associations present around the same label.

This case illustrates the operation of retrieval for one text. The accuracy tables quantify the decisions over the test collections, and the ablation table measures the contribution of changing specific pipeline components. These evidence types answer different questions. The example explains how a candidate appears in the classifier's input, while the aggregate results measure whether the resulting predictions match the required classes across the evaluation data.

4.9 Discussion and Chapter Summary

GORAG places structure between the observation of text and the construction of a language model's classification instruction. Keywords connect the current text to an accumulated graph. Edge costs and terminal connections select a subset of label alternatives. The model then classifies the original text with these alternatives and their representative information. Structure serves as a task-specific evidence-selection mechanism.

The online component changes the source of that evidence. Support examples establish supervised keyword–label associations. Subsequent predictions provide labels for newly observed query keywords, allowing the index to cover terms absent from the support sample. The offline ablation measures the contribution of this expansion under the evaluated conditions. Its bene-

fit is attached to a procedure whose state depends on earlier predictions, so the availability and ordering of query observations are part of the evaluation setting.

The graph's paths provide a trace of the index connections used for retrieval. They can reveal which keywords and labels were linked by the selected structure. This trace explains the provenance of candidate selection in the constructed graph. The final classifier still receives text and makes a language-model prediction; the retrieved path is an external evidence object rather than an observation of the model's internal computation.

Several conditions bound the interpretation of the chapter's evidence. Keyword extraction determines which parts of a query can become terminals. Sparse support data and limited label descriptions determine initial coverage. Predicted-label insertion can add incorrect associations. The accuracy tables use exact label matching and different backbones for different datasets. Each condition identifies a concrete component or evaluation choice that affects the reported classification result.

Within these conditions, the study shows how a task graph can reduce the label-selection burden placed on a language model and maintain lexical evidence as the task evolves. The contribution differs from KGLink's use of an existing knowledge graph to enrich table representations. Here the structure is built and updated from task data, and its immediate output is a query-specific label set. The next chapter turns from choosing among labels to selecting connected textual facts for an open-ended answer. That change requires a retrieval objective that accounts for the relevance and connection cost of evidence beyond a fixed classification vocabulary.

REFERENCES

[1] Y. Wang *et al.*, "GORAG: Graph-based online retrieval augmented generation for dynamic few-shot social media text classification," in *Proceedings of the ACM Web Conference 2026*, Association for Computing Machinery, 2026, pp. 9148–9159. DOI: 10.1145/3774904.3793011 (cit. on pp. 1, 2, 4, 16–18).

[2] C. Xia *et al.*, "Incremental few-shot text classification with multi-round new classes: Formulation, dataset and system," in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, K. Toutanova *et al.*, Eds., Online: Association for Computational Linguistics, 2021, pp. 1351–1360. DOI: 10.18653/v1/2021.naacl-main.106 (cit. on pp. 1, 9, 11).

[3] "Tf–idf," in *Encyclopedia of Machine Learning*, C. Sammut *et al.*, Eds. Boston, MA: Springer US, 2010, pp. 986–987. DOI: 10.1007/978-0-387-30164-8_832 (cit. on p. 4).

[4] K. Mehlhorn, "A faster approximation algorithm for the steiner problem in graphs," *Information Processing Letters*, vol. 27, no. 3, pp. 125–128, 1988. DOI: 10.1016/0020-0190(88)90066-X (cit. on p. 7).

[5] B. Vidgen *et al.*, "Introducing CAD: The contextual abuse dataset," in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, K. Toutanova *et al.*, Eds., Online: Association for Computational Linguistics, 2021, pp. 2289–2303. DOI: 10.18653/v1/2021.naacl-main.182 (cit. on p. 9).

[6] K. Kowsari *et al.*, "Hdltex: Hierarchical deep learning for text classification," in *2017 16th IEEE international conference on machine learning and applications (ICMLA)*, IEEE, 2017, pp. 364–371 (cit. on p. 9).

[7] H. Jiang *et al.*, "LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, L.-W. Ku *et al.*, Eds., Bangkok, Thailand: Association for Computational Linguistics, 2024, pp. 1658–1677. DOI: 10.18653/v1/2024.acl-long.91 (cit. on pp. 11, 16).

[8] D. Edge *et al.*, "From local to global: A graph rag approach to query-focused summarization," *arXiv preprint arXiv:2404.16130*, 2024 (cit. on pp. 11, 16).

[9] Z. Guo *et al.*, "LightRAG: Simple and fast retrieval-augmented generation," in *Findings of the Association for Computational Linguistics: EMNLP 2025*, C. Christodoulopoulos *et al.*, Eds., Suzhou, China: Association for Computational Linguistics, 2025, pp. 10 746–10 761. DOI: 10.18653/v1/2025.findings-emnlp.568 (cit. on pp. 11, 16).

[10] B. J. Gutiérrez *et al.*, "From RAG to memory: Non-parametric continual learning for large language models," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267, PMLR, 2025, pp. 21 497–21 515 (cit. on p. 11).

[11] A. Hurst *et al.*, "Gpt-4o system card," *arXiv preprint arXiv:2410.21276*, 2024 (cit. on p. 16).